

STUDENT NAME: **YASH CHAUHAN**

STUDENT ROLL NUMBER: **28**

SECTION: **CSE C-1**

STUDENT REG NUMBER: **230905194**

WEEK NO: **8**

DATE: **18 TH March 2026**

Lab No. 8 – Form Proessing using Django – Part-2 (Additional)

Q1. Create a website with two pages.

First page contains:

RadioButton with HP, Nokia, Samsung, Motorola, Apple as options.

CheckBox with Mobile and Laptop as items.

TextBox to enter quantity.

There is a button with text as "Produce Bill".

On Clicking Produce Bill button, item should be displayed with total amount on Another page.

Urls.py

```
from django.urls import path

from .views import bill_view, order_view

urlpatterns = [
    path('', order_view, name='order'),
    path('bill/', bill_view, name='bill'),
]
```

Urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('billing.urls')),
]
```

Forms.py

```
from django import forms

class BillForm(forms.Form):
    brand = forms.ChoiceField(
        label='Brand',
        choices=[
            ('HP', 'HP'),
            ('Nokia', 'Nokia'),
            ('Samsung', 'Samsung'),
            ('Motorola', 'Motorola'),
            ('Apple', 'Apple'),
        ],
        widget=forms.RadioSelect,
    )
    items = forms.MultipleChoiceField(
        label='Items',
        choices=[('Mobile', 'Mobile'), ('Laptop', 'Laptop')],
        widget=forms.CheckboxSelectMultiple,
    )
    mobile_quantity = forms.IntegerField(
        label='Mobile Quantity',
        min_value=0,
        required=False,
        initial=0,
    )
    laptop_quantity = forms.IntegerField(
        label='Laptop Quantity',
        min_value=0,
        required=False,
        initial=0,
    )

    def clean(self):
        cleaned_data = super().clean()
        items = cleaned_data.get('items', [])
        mobile_quantity = cleaned_data.get('mobile_quantity') or 0
        laptop_quantity = cleaned_data.get('laptop_quantity') or 0

        if not items:
            raise forms.ValidationError('Please select at least one item.')
```

```

if 'Mobile' in items and mobile_quantity < 1:
    self.add_error('mobile_quantity', 'Enter Mobile quantity for selected item.')

if 'Laptop' in items and laptop_quantity < 1:
    self.add_error('laptop_quantity', 'Enter Laptop quantity for selected item.')

return cleaned_data

```

Changes in setting.py

```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'billing',
]

```

Bill.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Bill</title>
<style>
body {
    font-family: Arial, sans-serif;
    margin: 20px;
}

.box {
    width: 460px;
    border: 1px solid #999;
    padding: 12px;
}

table {

```

```
width: 100%;
border-collapse: collapse;
margin-top: 8px;
}

th,
td {
border: 1px solid #bbb;
padding: 6px;
text-align: left;
font-size: 14px;
}

.total {
margin-top: 10px;
font-weight: bold;
}

a {
display: inline-block;
margin-top: 10px;
}
</style>
</head>
<body>
<div class="box">
<div><strong>Brand:</strong> {{ brand }}</div>

<table>
<thead>
<tr>
<th>Item</th>
<th>Unit Price</th>
<th>Quantity</th>
<th>Amount</th>
</tr>
</thead>
<tbody>
{% for row in line_items %}
<tr>
<td>{{ row.item }}</td>
<td>{{ row.unit_price }}</td>
<td>{{ row.quantity }}</td>
```

```

<td>{{ row.amount }}</td>
</tr>
{% endfor %}
</tbody>
</table>

<div class="total">Total Amount: {{ total_amount }}</div>
<a href="{% url 'order' %}">Back</a>
</div>
</body>
</html>

```

Order.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Produce Bill</title>
<style>
body {
font-family: Arial, sans-serif;
margin: 20px;
}

.box {
width: 360px;
border: 1px solid #999;
padding: 12px;
}

.group {
margin-bottom: 12px;
}

.group-title {
font-weight: bold;
margin-bottom: 6px;
}

.errorlist {
color: #b00020;

```

```
margin: 4px 0;
padding-left: 18px;
font-size: 13px;
}

input[type="number"] {
width: 120px;
padding: 3px;
}

button {
padding: 4px 10px;
font-size: 14px;
}
</style>
</head>
<body>
<div class="box">
<form method="post">
{% csrf_token %}

<div class="group">
<div class="group-title">Select Brand</div>
{{ form.brand.errors }}
{{ form.brand }}
</div>

<div class="group">
<div class="group-title">Select Item</div>
{{ form.items.errors }}
{{ form.items }}
</div>

<div class="group">
<div class="group-title">Quantities</div>
<div>
Mobile: {{ form.mobile_quantity }}
{{ form.mobile_quantity.errors }}
</div>
<div style="margin-top: 6px;">
Laptop: {{ form.laptop_quantity }}
{{ form.laptop_quantity.errors }}
</div>
```

```
</div>

<button type="submit">Produce Bill</button>
</form>
</div>
</body>
</html>
```



A screenshot of a web browser window displaying a form. The browser's address bar shows the URL "127.0.0.1:8000". The form is titled "Select Brand" and contains the following elements:

- Select Brand:** A list of radio buttons for "HP", "Nokia", "Samsung", "Motorola", and "Apple".
- Select Item:** A list of checkboxes for "Mobile" and "Laptop".
- Quantities:** Two input fields labeled "Mobile:" and "Laptop:", both containing the value "0".
- Produce Bill:** A button at the bottom of the form.

Brand: Apple			
Item	Unit Price	Quantity	Amount
Mobile	60000	12	720000
Laptop	120000	2	240000
Total Amount: 960000			
Back			

Q2. Write a program to display the following feedback form. The different options for the list box must be ASP-XML, DotNET, JavaPro and Unix,C,C++. When the Submit Form button is clicked after entering the data, a message as seen in the last line of the above figure must be displayed.

Urls.py

```
from django.urls import path

from .views import feedback_form_view

urlpatterns = [
    path("", feedback_form_view, name='feedback_form'),
]
```

Urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('feedback.urls')),
]
```



```
]
```

Views.py

```
from django.shortcuts import render

from .forms import FeedbackForm

def feedback_form_view(request):
    success_message = ""

    if request.method == 'POST':
        form = FeedbackForm(request.POST)
        if form.is_valid():
            name = form.cleaned_data['name']
            gender = form.cleaned_data['gender']
            course = form.cleaned_data['course']
            technical_coverage = form.cleaned_data['technical_coverage']
            success_message = (
                f'Thank you {name} ({gender}). Your feedback for {course} '
                f'with technical coverage rated as {technical_coverage} has been submitted successfully.'
            )
            form = FeedbackForm()
        else:
            form = FeedbackForm()

    return render(
        request,
        'feedback/feedback_form.html',
        {'form': form, 'success_message': success_message},
    )
```

Changes in setting.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
```

```
'django.contrib.messages',  
'django.contrib.staticfiles',  
'feedback',  
]
```

Forms.py

```
from django import forms  
  
class FeedbackForm(forms.Form):  
    name = forms.CharField(label='Name', max_length=100)  
    gender = forms.ChoiceField(  
        label='Gender',  
        choices=[('Male', 'Male'), ('Female', 'Female'), ('Other', 'Other')],  
        widget=forms.RadioSelect,  
    )  
    email = forms.EmailField(label='Email')  
    comments = forms.CharField(label='Comments', widget=forms.Textarea(attrs={'rows': 3}))  
    course = forms.ChoiceField(  
        label='Course',  
        choices=[  
            ('ASP-XML', 'ASP-XML'),  
            ('DotNET', 'DotNET'),  
            ('JavaPro', 'JavaPro'),  
            ('Unix,C,C++', 'Unix,C,C++'),  
        ],  
        widget=forms.Select,  
    )  
    technical_coverage = forms.ChoiceField(  
        label='Technical Coverage',  
        choices=[('Good', 'Good'), ('Bad', 'Bad'), ('Avg', 'Avg')],  
        widget=forms.RadioSelect,  
    )
```

Feed_back_form.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<title>Feedback Form</title>
```

```
<style>
body {
font-family: Arial, sans-serif;
margin: 20px;
}
```

```
.box {
width: 420px;
border: 1px solid #999;
padding: 12px;
}
```

```
.row {
margin-bottom: 10px;
}
```

```
label {
display: inline-block;
width: 130px;
vertical-align: top;
}
```

```
input,
textarea,
select {
width: 290px;
font-size: 14px;
padding: 4px;
box-sizing: border-box;
}
```

```
select {
height: 90px;
}
```

```
.errorlist {
color: #b00020;
margin: 4px 0;
padding-left: 18px;
font-size: 13px;
}
```

```
button {
```

```
padding: 4px 10px;
font-size: 14px;
}

.msg {
margin-top: 12px;
font-size: 14px;
}
</style>
</head>
<body>
<div class="box">
<form method="post">
{% csrf_token %}

<div class="row">
<label for="id_name">Name</label>
{{ form.name }}
{{ form.name.errors }}
</div>

<div class="row">
<label for="id_email">Email</label>
{{ form.email }}
{{ form.email.errors }}
</div>

<div class="row">
<label for="id_gender">Gender</label>
{{ form.gender }}
{{ form.gender.errors }}
</div>

<div class="row">
<label for="id_comments">Comments</label>
{{ form.comments }}
{{ form.comments.errors }}
</div>

<div class="row">
<label for="id_course">Course</label>
{{ form.course }}
{{ form.course.errors }}
```

```

</div>

<div class="row">
<label for="id_technical_coverage">Technical Coverage</label>
{{ form.technical_coverage }}
{{ form.technical_coverage.errors }}
</div>

<button type="submit">Submit Form</button>
</form>

{% if success_message %}
<div class="msg">{{ success_message }}</div>
{% endif %}
</div>
</body>
</html>

```

Name
qw

Email
q@gmail.com

Gender
☐ Male
☒ Female
☐ Other

Comments
NA

Course
DotNET

Technical Coverage
☐ Good
☒ Bad
☐ Avg

Submit Form

Thank you y (Other). Your feedback for DotNET with technical coverage rated as Bad has been submitted successfully.

Name

Email

Gender

Male

Female

Other

Comments

Course

ASP-XML

Technical Coverage

Good

Bad

Avg

Submit Form

Thank you y (Other). Your feedback for DotNET with technical coverage rated as Bad has been submitted successfully.